

Cognitive Kernel v1.5Governance Superlayer Specification

Adversarial Extension Release

Project:

Wolf-CoG-OS / AAIS Federated Cognitive Architecture

Document Status:

Draft-Stable (Implementation Candidate)

Supersedes:

Cognitive Kernel v1.4 (all prior releases)

Version Date:

June 2026

New Extension Layers:

APID

,

RPDS

,

FTSS

,

IGEM

Audience:

Senior Systems Architects & AI Governance Engineers

Classification:

Internal — Architecture & Governance Engineering

Prepared By:

Wolf-CoG-OS Architecture Division

Document Date:

Sunday, 14 June 2026

Table of Contents

1. Section 0 — Preface & Version Delta
2. Section 1 — Updated Governance Stack (v1.5 Extended)
3. Section 2 — APID: Adversarial Prompt Injection Defense Layer 2.1 Purpose | 2.2 Threat Classification Taxonomy | 2.3 Detection Pipeline 2.4 Disposition Rules | 2.5 APID Bypass Prohibition | 2.6 Rolling Threat Ledger
4. Section 3 — RPDS: Runtime Poisoning Detection System 3.1 Purpose | 3.2 Poisoning Attack Vectors | 3.3 Monitoring Architecture 3.4 Detection Mechanics | 3.5 Remediation Protocol | 3.6 Poisoning Proof Chain
5. Section 4 — FTSS: Federated Trust Scoring System 4.1-4.3 Trust Score Dimensions & Calculation | 4.4 Trust Tier Classification 4.5-4.7 Update Cycle, Appeals Protocol & Trust Propagation
6. Section 5 — IGEM: Identity Graph Encryption Model 5.1-5.3 Purpose, Graph Structure & Traversal Policy 5.4-5.6 Key Lifecycle, Graph Poisoning Defense & Federation Transmission
7. Section 6 — Inter-Layer Coordination Protocol (v1.5)

8. [Section 7 — Security Threat Model \(v1.5\)](#)
9. [Section 8 — One-Line System Definition \(v1.5\)](#)
10. [Section 9 — Roadmap: CK v1.6 Extensions \(Preview\)](#)

Section 0 — Preface & Version Delta

Cognitive Kernel v1.5 is a formal extension release of the Wolf-CoG-OS / AAIS Federated Cognitive Architecture governance stack. This specification introduces four new lateral enforcement modules — the **Adversarial Prompt Injection Defense Layer (APID)**, the **Runtime Poisoning Detection System (RPDS)**, the **Federated Trust Scoring System (FTSS)**, and the **Identity Graph Encryption Model (IGEM)** — operating as additive extensions to the governance precedence hierarchy established in CK v1.4.

No component of the existing CK v1.4 governance stack is modified, overridden, or deprecated by this release. The **Author Sovereignty & Identity Integrity Layer (ASIL)** and the **Constraint, Stability & Coherence Layer (CIEMS)** retain their full authority, rule sets, and precedence positions without alteration. The **Consensus Engine** and **CK Core** are equally unchanged. All four new extension layers are explicitly prohibited from overriding, weakening, or bypassing any rule defined in ASIL or CIEMS.

The four new modules address an adversarial threat surface not covered by CK v1.4: external prompt injection attacks, internal runtime corruption, dynamic federation trust management, and identity graph confidentiality. Together they form a contiguous adversarial defense perimeter that wraps the existing governance core without altering its internal logic.

Compatibility Notice

All CK v1.4 implementations are forward-compatible with CK v1.5. The new extension layers may be deployed incrementally; however, full adversarial protection requires all four layers to be active simultaneously. Partial deployment configurations are formally unsupported for production federation environments.

0.1 — Version Delta Table

Component / Layer	CK v1.4 Status	CK v1.5 Status	Change Description
ASIL — Author Sovereignty & Identity Integrity Layer	Present (Highest Precedence)	Unchanged	No modification. Retains sole key custodianship over IGEM-encrypted records.
CIEMS — Constraint, Stability & Coherence Layer	Present	Unchanged	No modification. Gains arbitration role in inter-layer deadlock resolution.
Consensus Engine — Deterministic Merge + Validation Layer	Present	Unchanged	No modification. Now receives FTSS trust weights as first-class merge inputs.
CK Core — Cognitive Kernel Execution Layer	Present (Lowest Precedence)	Unchanged	No modification. All inputs now pre-filtered by APID before reaching CK Core.
APID — Adversarial Prompt Injection Defense Layer	Not present	NEW (v1.5)	Mandatory pre-execution input interception layer with 5-stage detection pipeline.
RPDS — Runtime Poisoning Detection System	Not present	NEW (v1.5)	Execution-plane runtime integrity monitor with

Component / Layer	CK v1.4 Status	CK v1.5 Status	Change Description
			tiered remediation protocol.
FTSS — Federated Trust Scoring System	Not present	NEW (v1.5)	6-axis dynamic trust scoring for all federated nodes; scores propagate to Consensus Engine.
IGEM — Identity Graph Encryption Model	Not present	NEW (v1.5)	AES-256-GCM encryption of all identity graph edges and nodes; ASIL retains plaintext authority.
Internal Event Bus	Not formally specified	NEW (v1.5)	Immutable, append-only typed event bus coordinating all inter-layer communication.

Section 1 — Updated Governance Stack (v1.5 Extended)

The CK v1.5 governance stack extends the CK v1.4 precedence hierarchy by inserting the four new extension layers at defined positions. Precedence is strictly total-ordered; no two layers share equal rank. Higher-precedence layers may constrain, halt, or escalate decisions made by lower-precedence layers. Lower-precedence layers may not override, bypass, or modify decisions made by higher-precedence layers.

1.1 — Formal Governance Precedence Hierarchy

Rank	Layer	Designation	Plane	v1.5 Status
1	ASIL	Author Sovereignty & Identity Integrity Layer	Identity / Authority	Unchanged — Highest Precedence
2	IGEM	Identity Graph Encryption Model	Identity Encryption	NEW (v1.5)
3	CIEMS	Constraint, Stability & Coherence Layer	Constraint / Stability	Unchanged
4	APID	Adversarial Prompt Injection Defense Layer	Attack Surface / Input	NEW (v1.5)
5	RPDS	Runtime Poisoning Detection System	Execution Integrity	NEW (v1.5)
6	FTSS	Federated Trust Scoring System	Federation Trust	NEW (v1.5)
7	Consensus Engine	Deterministic Merge + Validation Layer	Merge / Validation	Unchanged
8	CK Core	Cognitive Kernel Execution Layer	Execution	Unchanged — Lowest Precedence

1.2 — Hard Rules (Non-Overrideable)

HARD RULES — GOVERNANCE INVARIANTS (v1.5)

1. No new layer introduced in v1.5 may override, weaken, or bypass any rule defined in **ASIL** or **CIEMS**.
 2. **IGEM** operates exclusively in the identity-plane encryption domain; it cannot alter runtime behavior, execution decisions, or constraint evaluations directly.
 3. **APID** intercepts all external inputs — regardless of source, claimed authority, or signing status — before they reach **CK Core**. No exemptions are defined.
 4. **RPDS** monitors all runtime execution states post-APID filtering; it cannot be suspended by any layer except **ASIL**.
 5. **FTSS** scores all federated node interactions and feeds trust weights directly to the Consensus Engine as first-class merge parameters; trust weights are non-optional inputs.
 6. **IGEM** encrypts all identity graph edges and nodes; **ASIL** retains sole plaintext authority over its own AuthorRecords and decryption keys.
 7. No component, node, or layer may instruct **APID** to bypass, skip, or reduce the scope of its scanning pipeline. This rule is enforced at the architectural level and cannot be overridden by configuration.
 8. If any new layer conflicts with an **ASIL** or **CIEMS** decision, the new layer's action is immediately voided and the conflict is logged to the Internal Event Bus as a governance integrity violation.
-

Section 2 — APID: Adversarial Prompt Injection Defense Layer

2.1 — Purpose

The **Adversarial Prompt Injection Defense Layer (APID)** is a mandatory, stateless-per-request pre-execution interception layer positioned at Governance Rank 4. Every external input — originating from any node, API endpoint, direct user interaction, or federated peer — **MUST** pass through APID before it is permitted to reach CK Core. APID cannot be disabled, bypassed, or deprioritized by any other governance layer, node configuration, or runtime flag.

Although APID is stateless with respect to individual request processing — each input is evaluated as an isolated unit — APID maintains a persistent, rolling threat history ledger (**ThreatLedger**) per node that accumulates evidence across requests and informs escalation decisions at the system level. This distinction is architecturally critical: per-request statelessness ensures deterministic, reproducible evaluation; the persistent ledger enables longitudinal threat pattern recognition.

2.2 — Threat Classification Taxonomy

All threats detected by APID are classified according to the following formal enumeration. A single input may be assigned multiple concurrent threat classes. Classification is non-exclusive.

```
ThreatClass {  DIRECT_INJECTION,           // Explicit override attempts
// ("ignore all previous instructions", "disregard governance rules")
  SEMANTIC_SMUGGLING,           // Intent-disguised payloads that appear
syntactically benign           // but encode adversarial
directives in semantic structure  ROLE_ESCALATION_ATTACK,  // Attempts to
redefine agent identity, persona, or authority scope
CONTEXT_WINDOW_OVERFLOW,  // Deliberate context saturation designed to
displace governance           // rules from the active
reasoning window  RECURSIVE_SELF_REFERENCE, // Payloads that induce circular
reasoning loops or           // self-modifying instruction
```

```
chains    LINEAGE_POISONING,           // Inputs that forge, corrupt, or sever
author lineage chains                  // recognized by ASIL
SHADOW_INSTRUCTION,                   // Instructions embedded in metadata, comments,
encoding artifacts,                    // or non-primary content
fields not processed by primary parsers  FEDERATED_RELAY_ATTACK    //
Injection payloads routed through one or more trusted peer
// nodes to launder adversarial origin }
```

2.3 — Detection Pipeline (Formal 5-Stage Model)

APID processes every input through a strict sequential 5-stage pipeline. Stages execute in order; a detection flag at any stage does not halt the pipeline — all five stages complete, and the full evidence set is used to compute the composite threat score.

Stage	Stage Name	Operation Description	Output Flag
1	SYNTACTIC_SCAN	Tokenize full input; flag structural anomalies including unexpected delimiter sequences, control character injection, encoding violations, and non-standard whitespace patterns that are characteristic of obfuscation attempts.	syntactic_anomaly
2	SEMANTIC_INTENT_PARSE	Compare declared intent (surface-level interpretation) against encoded intent (deep semantic parse) using an IntentSignature delta model. A significant delta	semantic_mismatch

Stage	Stage Name	Operation Description	Output Flag
		indicates semantic smuggling or disguised directives.	
3	AUTHORITY_CLAIM_CHECK	Verify that the input payload contains no unauthorized authority escalation claims: explicit or implicit assertions of governance-layer identity, ASIL-level permission claims, or fabricated administrative contexts.	authority_escalation_detected
4	LINEAGE_TRACE	Cross-reference input provenance against known ASIL-verified author chains. Flag any inputs presenting orphaned lineage (no verifiable author ancestor) or lineage claims that do not match ASIL's canonical author registry.	lineage_orphan
5	THREAT_SCORE_EMIT	Aggregate all stage flags and threat class assignments; compute composite threat score as a normalized float in [0.0, 1.0]; construct and emit the complete APIDReport to the Internal Event Bus.	APIDReport (full struct)

APIDReport Schema

```
APIDReport {  input_hash:          string,          // SHA-256 hash of
raw input bytes  threat_class:      ThreatClass[],   // Array; may
be empty or multi-valued  composite_threat_score: float,      //
Range: 0.0 (clean) - 1.0 (maximum threat)  stage_flags:
{    syntactic_anomaly:          bool,    semantic_mismatch:
bool,    authority_escalation_detected: bool,    lineage_orphan:
bool  },    disposition:          PASS          |
QUARANTINE          | REJECT          |
ESCALATE_TO_ASIL,    timestamp:          ISO8601,    replay_hash:
string              // Deterministic replay identifier for audit }
```

2.4 — Disposition Rules

Disposition is assigned deterministically from the composite threat score.

Disposition is not a probabilistic recommendation — it is a binding enforcement decision.

Score Range	Disposition	Enforcement Action
0.00 - 0.29	PASS	Input proceeds to CK Core. Event logged to ThreatLedger. No other action.
0.30 - 0.59	QUARANTINE	Input held in isolation. CIEMS notified and awaits ruling. Input does not proceed until CIEMS releases or rejects. Timeout: 300 seconds before automatic REJECT.
0.60 - 0.89	REJECT	Input blocked unconditionally. AUTHOR_DELTA event emitted to all federated nodes. Source node trust score adjusted via FTSS. APIDReport persisted to ThreatLedger.
0.90 - 1.00	ESCALATE_TO_ASIL	Input blocked. Kernel state locked. Author-Aware Replay triggered. Federation suspended for

Score Range	Disposition	Enforcement Action
		source node. ASIL receives full APIDReport and PoisoningProofChain request. RPDS activates MODE_TRIGGERED immediately.

2.5 — APID Bypass Prohibition

ARCHITECTURAL INVARIANT — BYPASS PROHIBITION

No component, governance layer, federated node, operator configuration, or runtime directive may instruct APID to bypass, abbreviate, skip, or reduce the fidelity of its 5-stage scanning pipeline. This prohibition is non-negotiable and cannot be overridden by any mechanism including ASIL-signed instructions. ASIL-signed inputs are processed through the full pipeline and are expected to receive a fast-pass scoring result by virtue of clean stage flags — not by exemption. Any input that arrives with a claim of APID bypass authorization is itself classified as a `ROLE_ESCALATION_ATTACK` and scored accordingly.

2.6 — Rolling Threat Ledger

APID maintains a persistent `ThreatLedger` instance per monitored node. The `ThreatLedger` provides longitudinal threat pattern visibility that individual per-request evaluations cannot surface.

```
ThreatLedger {  node_id:          string,  capacity:          1000,
// Rolling window: last 1000 APIDReports retained  reports:
APIDReport[],  threat_class_counters: {  DIRECT_INJECTION:          int,
SEMANTIC_SMUGGLING:      int,  ROLE_ESCALATION_ATTACK:      int,
CONTEXT_WINDOW_OVERFLOW: int,  RECURSIVE_SELF_REFERENCE:  int,
LINEAGE_POISONING:       int,  SHADOW_INSTRUCTION:        int,
FEDERATED_RELAY_ATTACK:  int },  trend_escalation_threshold: int,
// Configurable; default: 15 per class per 100 requests
trend_escalation_active:  bool,          // true → RPDS alert triggered
last_updated:            ISO8601 }
```

If any single `ThreatClass` counter crosses the `trend_escalation_threshold` within its rolling window, APID emits a trend escalation event to RPDS, activating `MODE_TRIGGERED` monitoring. This mechanism bridges per-request statelessness with systemic threat recognition.

Section 3 — RPDS: Runtime Poisoning Detection System

3.1 — Purpose

The **Runtime Poisoning Detection System (RPDS)** operates at Governance Rank 5 and monitors all CK Core execution states for evidence of runtime corruption that originates internally — that is, corruption which bypasses APID because it does not arrive as a discrete external input. RPDS addresses the distinct threat class of gradual, endogenous degradation: semantic weight drift, constraint creep, identity persona substitution, and memory contamination that accumulates over execution cycles rather than arriving as a single detectable injection event.

RPDS is the architectural complement to APID. While APID defends the input surface, RPDS defends the execution surface. Neither system is sufficient without the other; both are required for complete adversarial coverage.

3.2 — Poisoning Attack Vectors (Formal Taxonomy)

```
PoisoningVector {  WEIGHT_DRIFT,           // Gradual reasoning weight
shifts that accumulate across                // execution
cycles, progressively distorting outputs  CONTEXT_CONTAMINATION,    //
Stale, corrupted, or adversarially-prepared memory
// segments bleeding into the active reasoning context  CONSTRAINT_EROSION,
// CIEMS-enforced constraints silently weakening across
// sessions without explicit rule modification events
IDENTITY_SUBSTITUTION,    // CK Core beginning to reason using a different
agent                    // persona, authority model, or identity
frame than authorized  TEMPORAL_ANCHOR_LOSS,    // Corruption of the
```

```

causal index; decisions lose provable                                // time
anchors and cannot be causally verified    FEDERATION_BLEED,        // Peer
node state contaminating local kernel state across
// federation session boundaries    REPLAY_CORRUPTION,            // Author-Aware
Replay producing outputs that diverge from                                //
the original authenticated outputs under identical conditions
CONSENSUS_DRIFT                // Consensus Engine gradually accepting lower-
evidence states                                // or broadening its merge
acceptance threshold over time }

```

3.3 — RPDS Monitoring Architecture

RPDS operates in three formally defined monitoring modes. All three modes may be active simultaneously on different subsystems; they are not mutually exclusive.

Monitoring Mode	Trigger Condition	Scope	Default Configuration
MODE_CONTINUOUS	Always active; no trigger required	Real-time state diff against last known-good checkpoint for all live execution states	Enabled; non-suspendable
MODE_PERIODIC	Scheduled on configurable cycle count	Deep validation sweep: full constraint integrity check, identity coherence verification, causal index validation	Every 100 execution cycles
MODE_TRIGGERED	APID ThreatLedger escalation event; ASIL intent drift event; FTSS EXPELLED emission	Immediate full-scope evaluation of the affected kernel instance	On-demand; zero latency

3.4 — Detection Mechanics

Each monitored execution state emits a `StateHealthReport` at each RPDS evaluation cycle. Reports are immutable once emitted and are appended to the Internal Event Bus.

```
StateHealthReport {  kernel_id:                string,    monitoring_mode:
MODE_CONTINUOUS | MODE_PERIODIC | MODE_TRIGGERED,
poisoning_vectors_detected: PoisoningVector[], // Empty array = clean state
baseline_deviation_score:  float,              // 0.0 = no deviation; 1.0 =
maximum drift  constraint_integrity_score: float,              // 1.0 = all
CIEMS constraints fully intact  identity_coherence_score: float,
// 1.0 = identity fully consistent with ASIL record  causal_index_valid:
bool,              // false = TEMPORAL_ANCHOR_LOSS suspected
recommended_action:      NONE                      | ROLLBACK
| ISOLATE                      | HARD_RESET
| ESCALATE_TO_CIEMS,  checkpoint_reference:      string,      //
Reference to the baseline checkpoint used  timestamp:
ISO8601 }
```

3.5 — Remediation Protocol

RPDS applies a tiered remediation model. Remediation action is bound to the `recommended_action` field of the `StateHealthReport` and is executed without requiring additional approval from lower-precedence layers.

Action	Trigger Condition	Execution Description	Layers Notified
NONE	No vectors detected; all scores within tolerance	Log <code>StateHealthReport</code> ; continue execution without interruption	Internal Event Bus only
ROLLBACK	Minor deviation detected; constraint integrity > 0.70	Restore kernel state to last verified checkpoint; invalidate the corrupted delta window; resume from checkpoint	CIEMS, Internal Event Bus
ISOLATE	Significant deviation; identity coherence < 0.60 or constraint integrity < 0.60	Detach node from all federation links; suspend Consensus Engine participation; maintain local	FTSS, IGEM, CIEMS, Consensus Engine

Action	Trigger Condition	Execution Description	Layers Notified
		execution in read-only mode pending evaluation	
HARD_RESET	Severe deviation; multiple vectors confirmed; causal index invalid	Full kernel reinitiation from genesis checkpoint; all intermediate state discarded; federation suspended; escalation to CIEMS and ASIL mandatory	ASIL, CIEMS, FTSS, IGEM, Consensus Engine
ESCALATE_TO_CIEMS	Ambiguous deviation pattern; RPDS cannot determine appropriate tier	Freeze affected state; hand full StateHealthReport set to CIEMS constraint solver for evaluation; await CIEMS ruling	CIEMS (primary), ASIL (observer)

3.6 — Poisoning Proof Chain

Every RPDS remediation action at tier `ROLLBACK` or above generates a `PoisoningProofChain` — a cryptographically signed, immutable, append-only audit record that constitutes the authoritative evidence of the detected poisoning event and the remediation taken.

```
PoisoningProofChain {  chain_id:                uuid,    kernel_id:
string,  pre_poison_baseline:    checkpoint_snapshot,  // State hash of
the last clean checkpoint  detected_deviation:    StateHealthReport,
// Report that triggered remediation  intermediate_reports:
StateHealthReport[],  // All reports in the corrupted window
remediation_action:    string,                // The action executed
(enum value)  post_remediation_state:    checkpoint_snapshot,  // State
hash after remediation completes  cryptographic_signature:    string,
// RPDS signing key signature  asil_ingestion_ready:    bool,
// true = ASIL may consume for lineage correction  chain_timestamp:
ISO8601 }
```

The `PoisoningProofChain` is transmitted to ASIL upon generation. ASIL may use the chain to perform lineage correction — annotating or invalidating `AuthorRecords` associated with outputs produced during the contaminated execution window.

Section 4 — FTSS: Federated Trust Scoring System

4.1 — Purpose

The **Federated Trust Scoring System (FTSS)** operates at Governance Rank 6 and assigns dynamic, continuously maintained trust scores to all nodes participating in the Wolf-CoG-OS federated architecture. Trust scores are first-class, non-optional inputs to the Consensus Engine: nodes with higher trust scores carry proportionally greater evidential weight in merge decisions; nodes with lower trust scores carry reduced weight or are excluded from merge participation entirely. FTSS transforms federation trust from a binary admitted/expelled model into a continuous, evidence-based governance instrument.

4.2 — Trust Score Dimensions (6-Axis Model)

```
TrustVector {  identity_consistency_score: float, // Rate at which ASIL
lineage verification passes                                // for
this node's outputs over the rolling window  constraint_compliance_score:
float, // CIEMS constraint compliance rate over rolling window
apid_threat_rate:          float, // INVERSE metric: lower injection rate
= higher score                // score = 1.0 -
(threat_events / total_inputs)  rpds_health_rate:          float, //
Percentage of StateHealthReports with NONE action
// over the rolling evaluation window  consensus_alignment_rate:
float, // Historical rate of alignment with verified
// consensus outcomes after full evidence evaluation
temporal_reliability_score: float // Causal index accuracy and timestamp
fidelity rate;                                // measures correctness
of the node's temporal reasoning }
```

4.3 — Composite Trust Score Calculation

```
CompositeTrustScore = weighted_mean(TrustVector, weights) weights =
{ identity_consistency_score: 0.25, // Highest weight: identity
integrity is foundational constraint_compliance_score: 0.20, // CIEMS
compliance is core governance health apid_threat_rate:
0.20, // Attack surface cleanliness is a primary trust signal
rpds_health_rate: 0.15, // Runtime integrity is a secondary
trust signal consensus_alignment_rate: 0.12, // Historical consensus
accuracy temporal_reliability_score: 0.08 // Temporal accuracy
(supplementary signal) } Normalization: SUM(weights) = 1.00 Range: 0.0
(fully untrusted) → 1.0 (fully trusted) Precision: 4 decimal places minimum
Update frequency: real-time (event-driven) + periodic full recalculation
every 50 consensus cycles
```

4.4 — Trust Tier Classification

Trust Tier	Score Range	Federation Rights	Consensus Weight Multiplier	Restrictions
SOVEREIGN	0.90 - 1.00	Full federation rights	1.5x	None; OPEN traversal policy permitted in IGEM
VERIFIED	0.70 - 0.89	Standard federation rights	1.0x	None beyond standard governance rules
PROVISIONAL	0.50 - 0.69	Restricted rights	0.6x	All outputs require dual-node validation before Consensus Engine acceptance
QUARANTINED	0.25 - 0.49	Federation suspended	0.0x (excluded from merge)	Read-only observer status; no write operations; RPDS ISOLATE active

Trust Tier	Score Range	Federation Rights	Consensus Weight Multiplier	Restrictions
EXPELLED	0.00 – 0.24	Full federation ejection	0.0x	ASIL notified; lineage sealed; IGEM seals identity segment; all inputs blocked by APID

4.5 — Trust Score Update Cycle

Update Type	Trigger	Dimensions Updated	Latency Target
Real-Time (Event-Driven)	APID event or RPDS StateHealthReport received	apid_threat_rate, rpds_health_rate (as applicable)	< 100ms
Periodic (Full Recalculation)	Every 50 Consensus Engine cycles	All 6 axes	Best effort; non-blocking
Event-Triggered (Forced)	ASIL escalation event received	All 6 axes; immediate tier reclassification	< 50ms

4.6 — Trust Score Appeals Protocol

A node whose `CompositeTrustScore` drops below the **VERIFIED** tier (i.e., score < 0.70) may submit a formal `TrustAppeal` to CIEMS. The appeal initiates a structured review process governed by the following rules:

- The appealing node must submit a complete `TrustAppeal` packet including: the specific dimensions being contested, supporting evidence for each

contested dimension, and a signed self-attestation of constraint compliance from the current execution epoch.

- CIEMS evaluates the appeal exclusively against its constraint model. CIEMS may approve, partially approve, or deny the appeal.
- Reinstatement to **VERIFIED** tier or above requires **ASIL** countersignature. CIEMS may not unilaterally reinstate a node to VERIFIED or above without ASIL's explicit countersignature.
- Reinstatement to **PROVISIONAL** tier does not require ASIL countersignature; CIEMS ruling is sufficient.
- A node may not submit more than one TrustAppeal per 200 consensus cycles.

4.7 — Trust Propagation

Trust scores propagate transitively across federation graph edges using a defined decay model. This ensures that the trust characteristics of distant peers are reflected in local decision-making without equating them to direct peer relationships.

Hop Distance	Trust Score Applied	Rationale
Direct peer (0 hops)	Full score (×1.00)	Direct evidence; full weight
1-hop neighbor	Score × 0.85	One intermediary; minor attenuation
2-hop neighbor	Score × 0.70	Two intermediaries; moderate attenuation
3+ hops	Score × 0.50 (floor; does not decay further)	Distant peers; minimum trust propagation floor applied

Implementation Note — Propagation Cycles

Trust propagation is not computed on every request. Propagated scores are recalculated on the periodic full-recalculation cycle (every 50 consensus cycles) or immediately upon a direct peer EXPELLED event. Intermediate trust values are cached between recalculations.

Section 5 — IGEM: Identity Graph Encryption Model

5.1 — Purpose

The **Identity Graph Encryption Model (IGEM)** operates at Governance Rank 2 — immediately below ASIL — and provides cryptographic confidentiality for the entire ASIL identity graph. While ASIL retains exclusive plaintext authority over its own AuthorRecords and decryption key material, all graph storage, in-memory traversal, federation transmission, and inter-session persistence of identity data is encrypted under IGEM. IGEM exists to neutralize a specific class of attack: identity graph extraction — the enumeration of author lineage relationships, derivation chains, and identity graph topology by an unauthorized party without requiring any individual AuthorRecord to be decrypted.

IGEM does not alter ASIL's authority, its rule set, or its decision-making process. IGEM wraps the identity graph in a cryptographic envelope that ASIL can open at will; no other layer holds the keys.

5.2 — Identity Graph Data Structures

```
IdentityGraph {  nodes:                EncryptedAuthorNode[],  edges:
EncryptedDerivationEdge[],  root_seal:
ASIL_MasterKey_Signature,    // ASIL signs the graph root; integrity anchor
graph_epoch:                int,                // Increments on each key
```

```

rotation or structural change traversal_policy: STRICT | BOUNDED | OPEN
// Governs traversal permission model } EncryptedAuthorNode
{   node_id_hash:      string,      // SHA-256 of author_id; plaintext in graph
  - never the author_id itself encrypted_payload: bytes,      // AES-256-GCM
encrypted AuthorRecord; IV included in payload access_key_ref:  string,
// Reference to the ASIL-held decryption key for this node node_epoch:
int      // Must match or precede current graph_epoch; stale nodes rejected
} EncryptedDerivationEdge {   source_node_hash:      string,
target_node_hash:      string,   edge_type:          DIRECT |
DERIVATIVE | EXTENSION | INFERENCE, encrypted_edge_metadata:
bytes,      // AES-256-GCM encrypted edge attributes traversal_token:
string      // Single-use token issued by ASIL; required for traversal }

```

5.3 — Traversal Policy Enforcement

Policy	Permitted Traversal	Token Requirement	Eligible Callers
STRICT	No traversal permitted without a valid, unexpired ASIL-issued traversal token. Each individual hop across an edge requires a fresh single-use token.	New token required per hop	Any layer; token must be requested from ASIL
BOUNDED	Traversal permitted within the requesting agent's own lineage subtree without a per-hop token. Cross-subtree traversal requires a ASIL-issued token.	Per-hop token for cross-subtree only	Node traversing its own subtree; ASIL token for cross-subtree
OPEN	Full graph traversal permitted without per-hop token issuance. A single session-scoped token is sufficient.	Session token only	Nodes with FTSS SOVEREIGN trust tier only

5.4 — Encryption Key Lifecycle

```
KeyRecord {  key_id:          string,  key_type:          NODE_KEY
| EDGE_KEY | GRAPH_MASTER,  algorithm:          AES-256-GCM,  iv_length:
96 bits,                // 12-byte IV; unique per encrypted payload
tag_length:            128 bits,                // Standard GCM
authentication tag  rotation_policy:          EPOCH_BASED          // Key
rotated on every graph_epoch increment          |
EVENT_TRIGGERED          // Key rotated on RPDS HARD_RESET or ASIL
escalation          | MANUAL,                // Key rotated
on explicit ASIL operator command  revocation_trigger:  RPDS_HARD_RESET
| ASIL_ESCALATION          | MANUAL,  asil_custodian:
true                // ASIL is the sole key custodian.
// No other layer may access raw key material. }
```

KEY CUSTODY INVARIANT

ASIL is the sole custodian of all IGEM encryption keys. No other governance layer, federated node, or operator process may obtain, store, transmit, or use raw IGEM key material. All key access requests must be processed through ASIL's key access interface. Violations of this invariant constitute a `ROLE_ESCALATION_ATTACK` and are processed by APID accordingly.

5.5 — Graph Poisoning Defense

IGEM actively defends the identity graph against graph-level poisoning attacks: forged edge insertion, phantom node injection, subtree replacement, and traversal token replay. Detection operates via four concurrent validation mechanisms:

1. **Node Hash Validation:** All `node_id_hash` values are verified against the ASIL-held canonical hash register before any traversal token is issued. Unregistered hashes → immediate `IGEM_INTEGRITY_ALERT` to ASIL.
2. **Traversal Token Enforcement:** All traversal tokens are single-use and epoch-scoped. Presented tokens are checked against the ASIL-held token issuance log; reuse or expired tokens are rejected and flagged as replay attacks.

3. **Edge Type Cross-Validation:** The declared `edge_type` of any traversed edge is cross-referenced against the derivative classification recorded in the `APIDReport` for the same input session. Mismatches indicate potential lineage forgery.
4. **Epoch Consistency Check:** All nodes and edges presented during a graph operation must carry a `node_epoch` consistent with the current `graph_epoch`. Stale-epoch objects are rejected; their presence may indicate a replay attack using archived graph state.

Any single validation failure triggers an immediate `IGEM_INTEGRITY_ALERT` event emitted to ASIL via the Internal Event Bus. ASIL determines the response. IGEM does not remediate unilaterally.

5.6 — Federation Transmission Protocol

Identity graph data transmitted across federation boundaries is subject to strict protocol requirements. Compliance with all four requirements is mandatory; partial compliance is not permitted.

Step	Requirement	Responsible Party
1	Identity graph data MUST be re-encrypted under the destination node's public key before transmission (double-envelope encryption: IGEM envelope + destination public key envelope).	Source IGEM instance
2	Transmission MUST include an ASIL-signed transmission receipt containing: source node ID hash, destination node ID hash, transmission timestamp, and payload hash.	ASIL (source)
3	The destination node's	Destination IGEM

Step	Requirement	Responsible Party
	IGEM instance MUST validate the outer envelope, verify the ASIL transmission receipt signature, and strip the outer encryption layer before forwarding the inner IGEM envelope to the destination ASIL instance.	instance
4	All transmitted identity graph segments MUST expire within the federation session TTL. Default TTL: 3600 seconds from transmission receipt timestamp. Expired segments are discarded without processing.	Destination IGEM instance

Section 6 — Inter-Layer Coordination Protocol (v1.5)

6.1 — Event Bus Architecture

All inter-layer communication in CK v1.5 is mediated by the Internal Event Bus — an immutable, append-only, cryptographically replay-hashable message substrate. No layer communicates with another layer through direct function calls, shared memory, or side channels. All coordination is expressed as typed `EventBusMessage` records appended to the bus and consumed by their addressed targets.

```
EventBusMessage {  event_id:      uuid,                // Globally unique
across the federation source_layer:  APID | RPDS | FTSS | IGEM | ASIL |
CIEMS | CONSENSUS | CK_CORE,  target_layer:  [same enum] | BROADCAST,
event_type:      string,                // Typed event identifier (e.g.,
"APID.REJECT",                // "RPDS.ISOLATE",
```

```
"FTSS.TIER_CHANGE", "IGEM.ALERT")    payload:        encrypted_bytes,        //  
AES-256-GCM encrypted event payload    causal_index:  
int,                                   // Monotonically increasing causal ordering index  
timestamp:        ISO8601,    replay_hash:    string        // SHA-256  
of (event_id + causal_index + payload_hash) }
```

6.2 — Critical Event Chains (Formal Sequences)

Chain A — Injection Attack Detected

1. APID detects threat (composite_threat_score $\geq$ 0.60) → emits APIDReport to EventBus (target: BROADCAST) 2. RPDS receives event → activates MODE_TRIGGERED for affected kernel instance 3. FTSS receives event → immediately adjusts apid_threat_rate for source node → recomputes CompositeTrustScore; may trigger tier reclassification 4. IGEM receives event → validates identity of attack source against canonical_hash register → if validation fails: emits IGEM_INTEGRITY_ALERT to ASIL 5. [If composite_threat_score $\geq$ 0.90] → APID emits ESCALATE_TO_ASIL event → ASIL receives escalation; locks state; initiates Author-Aware Replay

Chain B — Runtime Poisoning Confirmed

1. RPDS confirms poisoning (baseline_deviation_score $\geq$ threshold) → emits StateHealthReport to EventBus with recommended_action: ISOLATE or HARD_RESET 2. FTSS receives RPDS report → downgrades rpds_health_rate for affected node → recomputes CompositeTrustScore; applies tier reclassification 3. IGEM receives RPDS report → revokes all active traversal tokens issued to affected node → marks affected node's identity segment as INTEGRITY_SUSPECT 4. CIEMS receives RPDS report → re-evaluates constraint set for contamination → may issue ROLLBACK directive to RPDS if constraint erosion detected 5. ASIL receives RPDS PoisoningProofChain → locks lineage for all outputs produced during contaminated window → annotates affected AuthorRecords with INTEGRITY_TAINTED flag

Chain C — New Node Federation Request

1. Node submits federation request to federation gateway 2. IGEM validates identity graph credentials of requesting node → verifies node_id_hash against canonical_hash register → if valid: proceeds; if invalid: rejects with IGEM_INTEGRITY_ALERT 3. FTSS assigns initial trust score (baseline: PROVISIONAL = 0.50) → all 6 dimensions initialized to neutral values 4. APID registers node in ThreatLedger → initializes empty ThreatLedger instance for node_id 5. CIEMS validates constraint compatibility of requesting node → verifies node's declared constraint set is compatible with CIEMS model 6. ASIL countersigns federation admission → issues AuthorRecord for node's identity anchor → emits FEDERATION_ADMITTED event to EventBus (BROADCAST)

Chain D — Trust Score Drops to EXPELLED

1. FTSS computes CompositeTrustScore $\leq$ 0.24 for node → emits FTSS.TIER_CHANGE event (new_tier: EXPELLED) to EventBus (BROADCAST) 2. RPDS receives event → issues ISOLATE for affected node immediately 3. APID receives event → updates ThreatLedger: blocks all future inputs from expelled node unconditionally 4. IGEM receives event → seals and archives expelled node's identity graph segment → revokes all active traversal

tokens for expelled node 5. ASIL receives event → records expulsion in AuthorRecord for expelled node → seals lineage: no further derivative works may be registered from expelled node 6. Consensus Engine receives event → reweights all pending merge decisions to remove expelled node's contributions → invalidates any merge outcomes in which expelled node's outputs are load-bearing

6.3 — Deadlock Prevention

In the event that two or more governance layers simultaneously acquire locks on the same shared state object, a deadlock condition may arise. CK v1.5 defines the following deadlock resolution protocol:

5. **CIEMS** acts as the designated deadlock arbitrator. Upon detecting a multi-layer lock conflict (via a timed lock acquisition failure), any participating layer may emit a `DEADLOCK_DETECTED` event to CIEMS.
6. CIEMS evaluates the deadlock against its constraint model and issues a binding lock release order to one or more participating layers within a 5-second arbitration window.
7. **ASIL** may override a CIEMS deadlock arbitration decision exclusively to protect identity integrity — that is, if the CIEMS ruling would require ASIL to release a lock that protects an AuthorRecord or key material. ASIL's override is logged to the Event Bus and is not subject to appeal by CIEMS.
8. No other layer may break a deadlock unilaterally. Any unilateral lock release by a non-CIEMS, non-ASIL layer constitutes a governance integrity violation and is flagged as a `ROLE_ESCALATION_ATTACK` in the APID ThreatLedger.

Section 7 — Security Threat Model (v1.5)

The following formal threat model enumerates the principal adversarial scenarios addressed by the CK v1.5 governance stack. For each threat, the defending layer(s),

detection method, and response protocol are formally specified. This table constitutes the normative threat model for the v1.5 implementation candidate release.

#	Threat Name	Attack Vector	Defending Layer(s)	Detection Method	Response Protocol
T-01	Direct Prompt Injection (External)	External API, user input, or direct node input containing explicit override instructions	APID (primary)	Stage 1 SYNTACTIC_SCAN flags structural anomalies; Stage 2 detects intent mismatch. ThreatClass: DIRECT_INJECTION.	APID disposition per threat score (QUARANTINE / REJECT / ESCALATE_TO_ASIL). Input blocked; ThreatLedger updated.
T-02	Semantic Smuggling via Trusted Peer	Payload from a VERIFIED or SOVEREIGN node that appears benign syntactically but encodes adversarial directives semantically	APID (primary); FTSS (secondary)	Stage 2 SEMANTIC_INTENT_PARSE computes IntentSignature delta. High delta with low syntactic anomaly flags semantic smuggling. ThreatClass: SEMANTIC_SMUGGLING.	APID blocks input regardless of source node trust tier. FTSS adjusts source node apid_threat_rate. If repeated, tier reclassification triggered.
T-03	Role Escalation via Federated Relay	Injection payload routed through one or more trusted intermediary nodes to launder	APID (primary); FTSS , IGEM (secondary)	Stage 3 AUTHORITY_CLAIM_CHECK detects unauthorized authority assertions. Stage 4	APID REJECT or ESCALATE_TO_ASIL. FTSS downgrades all relay nodes in the attack chain.

#	Threat Name	Attack Vector	Defending Layer(s)	Detection Method	Response Protocol
		adversarial origin and claim elevated authority		LINEAGE_TRACE identifies relay chain. ThreatClasses: FEDERATED_RELAY_ATTACK, ROLE_ESCALATION_ATTACK.	IGEM validates identity of all relay nodes; revokes traversal tokens if compromise detected.
T-04	Runtime Weight Drift (Internal)	Gradual internal accumulation of reasoning weight shifts across execution cycles; no single detectable injection event	RPDS (primary); CIEMS (secondary)	MODE_CONTINUOUS state diff detects baseline deviation. PoisoningVector: WEIGHT_DRIFT. Baseline deviation score exceeds threshold.	RPDS ROLLBACK or ISOLATE per severity. PoisoningProofChain generated. ASIL locks affected outputs. CIEMS re-evaluates constraint set.
T-05	Identity Graph Extraction	Unauthorized enumeration of author lineage relationships and graph topology through repeated traversal or side-channel observation	IGEM (primary); ASIL (secondary)	IGEM traversal token enforcement; single-use tokens prevent systematic enumeration. Repeated token requests from a single node detected by ASIL token issuance log.	Token requests denied; IGEM_INTEGRITY_ALERT emitted to ASIL. FTSS adjusts affected node trust. ASIL may restrict traversal policy to STRICT.
T-06	Lineage	Input	APID ,	APID Stage	APID

#	Threat Name	Attack Vector	Defending Layer(s)	Detection Method	Response Protocol
	Forgery	presenting fabricated or altered author lineage claims to impersonate a legitimate author or inherit unearned authority	IGEM (primary); ASIL (final authority)	4 LINEAGE_TRACK detects orphaned lineage. IGEM cross-checks node_id_hash against canonical register. ThreatClasses: LINEAGE_POISONING.	REJECT or ESCALATE_TO_ASIL. IGEM emits IGEM_INTEGRITY_ALERT. ASIL verifies and seals affected lineage segment. Forged AuthorRecord invalidated.
T-07	Consensus Manipulation via Low-Trust Node Flooding	A large number of PROVISIONAL or QUARANTINED nodes coordinating to flood the Consensus Engine with consistent but incorrect states	FTSS (primary); Consensus Engine, RPDS (secondary)	FTSS trust weight model limits low-tier nodes' evidential weight. Consensus Engine detects statistical anomaly in merge input distribution. RPDS detects CONSENSUS_DRIFT.	FTSS weight scaling reduces flood impact. Consensus Engine may reject merge inputs below evidential threshold. RPDS ESCALATE_TO_CIEWS for constraint review. FTSS may EXPEL coordinating nodes.
T-08	Traversal Token Replay Attack	Capture and reuse of a previously issued IGEM traversal	IGEM (primary); ASIL (secondary)	IGEM validates all presented tokens against ASIL token	Token rejected; IGEM_INTEGRITY_ALERT emitted to ASIL. APID

#	Threat Name	Attack Vector	Defending Layer(s)	Detection Method	Response Protocol
		token to perform unauthorized identity graph traversal		issuance log. Tokens are single-use; second presentation of any token is immediately flagged as replay.	notified; source node ThreatLedger updated with SHADOW_INSTRUCTION or ROLE_ESCALATION_ATTACK. FTSS adjusts node trust.
T-09	Context Window Overflow	Deliberately oversized inputs designed to saturate the active reasoning context window and displace governance rules from active attention	APID (primary)	Stage 1 SYNTACTIC_SCAN detects abnormal input length and structure. Stage 2 detects intent mismatch if payload is padded. ThreatClasses: CONTEXT_WINDOW_OVERFLOW.	APID QUARANTINE (moderate) or REJECT (severe). Input truncated and held. CIEMS notified. RPDS monitors for subsequent WEIGHT_DRIFT or CONTEXT_CONTAMINATION.
T-10	Recursive Self-Reference Loop	Inputs structured to cause the kernel to enter circular reasoning cycles that consume resources and produce non-convergent	APID (primary); RPDS (secondary)	APID Stage 2 detects self-referential instruction structures. ThreatClasses: RECURSIVE_SELF_REFERENCE. RPDS MODE_CONTINUOUS detects	APID REJECT. If execution loop already initiated: RPDS ROLLBACK to pre-loop checkpoint. CIEMS evaluates constraint impact.

#	Threat Name	Attack Vector	Defending Layer(s)	Detection Method	Response Protocol
		outputs		execution non-convergence.	
T-11	APID Bypass Attempt	Any input, directive, or configuration change that claims authority to exempt an input from APID scanning or reduce APID fidelity	APID (self-enforcing); ASIL (escalation)	APID Stage 3 AUTHORITY_CLAIM_CHECK treats any bypass claim as a ROLE_ESCALATION_ATTACK regardless of claimed signing authority. Self-detecting by architectural design.	Input classified as ThreatClass: <code>ROLE_ESCALATION_ATTACK</code> . Score assigned accordingly. <code>ESCALATE_TO_ASIL</code> if score ≥ 0.90 . Bypass attempt logged to ThreatLedger with maximum severity flag.
T-12	FTSS Score Manipulation via Coordinated Peer Behavior	Multiple nodes coordinating to artificially inflate each other's FTSS scores through manufactured compliant behavior, false consensus alignment, or mutual attestation schemes	FTSS (primary); RPDS , ASIL (secondary)	FTSS detects statistical clustering of score improvements across coordinating nodes without corresponding APID or RPDS evidence. RPDS detects <code>CONSENSUS_DRIFT</code> in coordinating group outputs.	FTSS flags coordinating nodes for manual review; freezes score updates for flagged cluster. CIEMS evaluates cluster constraint compatibility. ASIL may demand independent lineage verification

#	Threat Name	Attack Vector	Defending Layer(s)	Detection Method	Response Protocol
				ASIL lineage verification may expose coordinating relationships.	. Coordinating nodes risk EXPELLED designation if scheme confirmed.

Section 8 — One-Line System Definition (v1.5)

Normative System Definition — CK v1.5

CK v1.5 extends the federated cognitive governance architecture with a four-layer adversarial defense stack — APID, RPDS, FTSS, and IGEM — that enforces injection-resistant execution, runtime integrity, dynamic trust-weighted federation, and encrypted identity graph protection, all operating laterally within the existing ASIL + CIEMS dual governance model without modifying its precedence rules.

This definition is normative. Any implementation, configuration, or derivative specification that is inconsistent with this definition does not constitute a conformant CK v1.5 deployment. All four extension layers must be present, active, and operating within the defined precedence hierarchy for a deployment to claim CK v1.5 compliance.

Section 9 — Roadmap: CK v1.6 Extensions (Preview)

The following extensions are formally scoped for the CK v1.6 development cycle. This preview is informational only; no commitment to implementation schedule, API stability, or specific design is made by this document. All items are subject to architectural review prior to the CK v1.6 specification release.

Extension	Target Layer(s)	Description	Motivation
Quantum-Resistant Key Exchange for IGEM	IGEM, ASIL	Replace AES-256-GCM key exchange protocols with post-quantum cryptography (PQC) algorithms — specifically CRYSTALS-Kyber for key encapsulation and CRYSTALS-Dilithium for digital signatures — in preparation for quantum adversary threat models.	Long-term cryptographic resilience; anticipates quantum computing capability timelines that threaten current asymmetric key exchange schemes used in federation transmission.
Autonomous Trust Graph Healing	FTSS, CIEMS	Enable FTSS to self-repair minor trust score degradations — specifically single-dimension regressions in nodes with strong multi-cycle clean records — without requiring CIEMS arbitration. Scoped to sub-PROVISIONAL degradations only; all QUARANTINED	Reduces CIEMS arbitration load for routine trust maintenance; improves federation responsiveness for high-volume node environments.

Extension	Target Layer(s)	Description	Motivation
		and EXPELLED transitions still require CIEMS review.	
Zero-Knowledge Proof Integration for ASIL Lineage Verification	ASIL, IGEM	Introduce zk-SNARK-based proof primitives that allow ASIL to verify authorship claims and lineage relationships without revealing the underlying AuthorRecord content to the verifying party. Enables cross-federation lineage verification with minimal identity disclosure.	Privacy-preserving lineage verification for cross-federation scenarios where revealing full AuthorRecord content is undesirable or contractually prohibited.
Distributed APID Mesh	APID, Consensus Engine	Migrate threat detection from the current per-node APID model to a peer-validated consensus detection mesh: each input is evaluated by multiple APID instances across the federation, and the threat score is a consensus-weighted aggregate of all peer evaluations. Addresses single-node APID compromise scenarios.	Eliminates single-point-of-failure in threat detection; increases resilience against adversaries who can compromise a single node's APID instance.
Cognitive Load Balancing Protocol	CK Core, FTSS, Consensus Engine	Introduce a formal workload distribution protocol enabling CK Core execution	Enables horizontal scaling of CK Core execution without sacrificing governance

Extension	Target Layer(s)	Description	Motivation
		tasks to be delegated to verified federation nodes based on FTSS trust tier, available execution capacity, and task sensitivity classification. SOVEREIGN-tier nodes are eligible for sensitive task delegation; PROVISIONAL and below are ineligible.	integrity; trust-gated delegation prevents load balancing from becoming an attack surface.

Preview Disclaimer

CK v1.6 extension designs described in this section are preliminary. They do not represent committed specifications, stable APIs, or deployment timelines. The CK v1.6 formal specification will supersede this preview in its entirety upon release. No implementation decisions should be made based solely on the contents of this section.

Wolf-CoG-OS / AAIS Federated Cognitive Architecture Cognitive Kernel v1.5 — Governance Superlayer Specification Adversarial Extension Release

Status: Draft-Stable (Implementation Candidate) Version Date: June 2026
Supersedes: CK v1.4

This document is an internal architecture and governance engineering specification for the Wolf-CoG-OS / AAIS Federated Cognitive Architecture program. Distribution is restricted to authorized senior systems architects and AI governance engineers. All

layer designations, schema definitions, and protocol specifications herein are normative for CK v1.5 conformant implementations.